

Quantum Key Distribution with BB84 in QuNetSim

A Practical Implementation Using the QuNetSim Quantum Network Simulator

Randy Balzer
March 16th, 2026

Agenda

- Introduction to the BB84 Protocol
- Overview of QuNetSim
- Prerequisites and Installation of QuNetSim
- Network Topology and Setup
- Key Generation and Protocol Parameters
- Alice's Protocol – Encoding and Transmission
- Eve's Protocol – Measurement and Key Collection
- Key Conversion and Illustrative Encryption
- Message Transmission and Reception
- Main Execution
- Conclusion

Introduction to the BB84 Protocol

- Developed by Charles Bennett and Gilles Brassard in 1984
- Shared secret over an insecure quantum channel detecting potential eavesdroppers
- Leverages the principles of quantum mechanics: no-cloning theorem and uncertainty principle

Overview of QuNetSim

- Python-based framework specifically designed for simulating quantum networks at the application and network layers
- Purpose-built for quantum communication protocols
- High-level abstraction
- Support for hybrid classical–quantum communication
- Extensibility and flexibility

Prerequisites and Installation of QuNetSim

QuNetSim requires

- Python 3.6 or higher
- A virtual environment is strongly recommended

Installation of QuNetSim

Installation via pip (recommended)

- Execute in terminal

```
pip install qunetsim
```

Installation from source

- Clone the repository

```
git clone https://github.com/tqsd/QuNetSim.git
```

- Create/activate a virtual environment

```
python -m venv venvsource venv/bin/activate (Linux/  
macOS)
```

```
venv\Scripts\activate (Windows)
```

- Install dependencies

```
pip install -r requirements.txt
```

- Set PYTHONPATH

```
export PYTHONPATH=$PYTHONPATH:$PWD (adjust path as  
needed)
```

After installation, import statements such as `from qunetsim.components import Network, Host` should function correctly

Network Topology and Setup

Topology

- The simulation uses a linear network:

Alice <--> Bob <--> Eve

- Bob is a relay only, does not interfere with the qubits

```
import numpy as np
import random
from qunetsim.components import Host, Network
from qunetsim.objects import Qubit

network = Network.get_instance()
network.start(['Alice', 'Bob', 'Eve'])
network.delay = 0.0

host_alice = Host('Alice')
host_alice.add_connection('Bob')
host_alice.start()

host_bob = Host('Bob')
host_bob.add_connection('Alice')
host_bob.add_connection('Eve')
host_bob.start()

host_eve = Host('Eve')
host_eve.add_connection('Bob')
host_eve.start()

network.add_hosts([host_alice, host_bob, host_eve])
```

Key Generation and Protocol Parameters

- A random secret key of fixed length is generated by Alice

```
key_size = 10  
secret_key = np.random.randint(2, size=key_size)
```

- Each bit is independently encoded in either the Z-basis or X-basis
- Eve measures in a randomly chosen basis
- Only matching bases contribute to the final key

Alice's Protocol – Encoding and Transmission

Alice

- Prepares qubits
- Transmits them
- Performs basis sifting via classical acknowledgments.

```
def alice_qkd(alice, msg_buffer, secret_key, receiver):
    sequence_nr = 0
    for bit in secret_key:
        ack = False
        while not ack:
            base = random.randint(0, 1)          # 0 = Z, 1 = X
            q = Qubit(alice)
            if bit == 1:
                q.X()
            if base == 1:
                q.H()
            alice.send_qubit(receiver, q, await_ack=True)
            msg = alice.get_message(receiver, msg_buffer, sequence_nr)
            if msg == f"{sequence_nr} : {base}":
                ack = True
                alice.send_classical(receiver, f"{sequence_nr} :0", await_ack=True)
            else:
                alice.send_classical(receiver, f"{sequence_nr} :1", await_ack=True)
        sequence_nr += 1
```

Eve's Protocol – Measurement and Key Collection

Eve

- Measures incoming qubits
- Coordinates with Alice to form the shared key

```
def eve_qkd(eve, msg_buffer, key_size, sender):
    sequence_nr = 0
    received_count = 0
    key_array = []
    while received_count < key_size:
        base = random.randint(0, 1)
        q = eve.get_qubit(sender, wait=10)
        while q is None:
            q = eve.get_qubit(sender, wait=10)
        if base == 1:
            q.H()
        bit = q.measure()
        eve.send_classical(sender, f"{sequence_nr} : {base}", await_ack=True)
        ack = eve.get_message(sender, msg_buffer, sequence_nr)
        if ack == f"{sequence_nr} :0":
            received_count += 1
            key_array.append(bit)
            sequence_nr += 1
    return key_array
```

Key Conversion and Simple Encryption

```
def key_array_to_string(key_array):  
    binary = ''.join(str(b) for b in key_array)  
    chars = [chr(int(binary[i:i+8], 2)) for i in range(0, len(binary), 8)]  
    return ''.join(chars)  
  
def xor_encrypt_decrypt(key_str, text):  
    return ''.join(chr(ord(k) ^ ord(c)) for k, c in zip(key_str * (len(text) // len(key_str) + 1), text))
```

Important - XOR scheme is used for demonstration only and is not cryptographically secure.

Message Transmission and Reception

- Alice encrypts and sends a test message
- Eve decrypts it using the shared key

```
def alice_send_message(alice, key_array, receiver):  
    msg = "Hi Eve, how are you???"  
    key_str = key_array_to_string(key_array)  
    encrypted = xor_encrypt_decrypt(key_str, msg)  
    alice.send_classical(receiver, f"-1:{encrypted}", await_ack=True)
```

```
def eve_receive_message(eve, msg_buffer, key_array, sender):  
    raw = eve.get_message(sender, msg_buffer, -1)  
    encrypted = raw.split(':', 1)[1]  
    key_str = key_array_to_string(key_array)  
    decrypted = xor_encrypt_decrypt(key_str, encrypted)  
    print(f"Eve received decoded message: {decrypted}")
```


Main Execution and Threading

- Protocols run concurrently in threads for Alice and Eve

```
def alice_func(alice):  
    msg_buff = []  
    alice_qkd(alice, msg_buff, secret_key, host_eve.host_id)  
    alice_send_message(alice, secret_key, host_eve.host_id)  
  
def eve_func(eve):  
    msg_buff = []  
    eve_key = eve_qkd(eve, msg_buff, key_size, host_alice.host_id)  
    eve_receive_message(eve, msg_buff, eve_key, host_alice.host_id)  
  
t1 = host_alice.run_protocol(alice_func, ())  
t2 = host_eve.run_protocol(eve_func, ())  
t1.join()  
t2.join()
```

```
Alice sent 1 key bits  
Eve received 1 key bits.  
Alice sent 2 key bits  
Eve received 2 key bits.  
Alice sent 3 key bits  
Eve received 3 key bits.  
Alice sent 4 key bits  
Alice sent 4 key bits  
Alice sent 4 key bits  
Alice sent 4 key bits  
Alice sent 5 key bits  
Alice sent 6 key bits  
Alice sent 6 key bits  
Alice sent 7 key bits  
Alice sent 7 key bits  
Alice sent 8 key bits  
Alice sent 8 key bits  
Alice sent 9 key bits  
Alice sent 9 key bits  
Alice sent 9 key bits  
Alice sent 9 key bits  
Alice sent 10 key bits  
Alice sends encrypted message
```

Conclusion

- Implementation illustrates the core mechanics of BB84 within a network-aware simulator
- QuNetSim enables straightforward extension to multi-node topologies

For production-grade QKD, real-world deployments

- Must incorporate parameter estimation
- Error correction
- Privacy amplification

Questions?